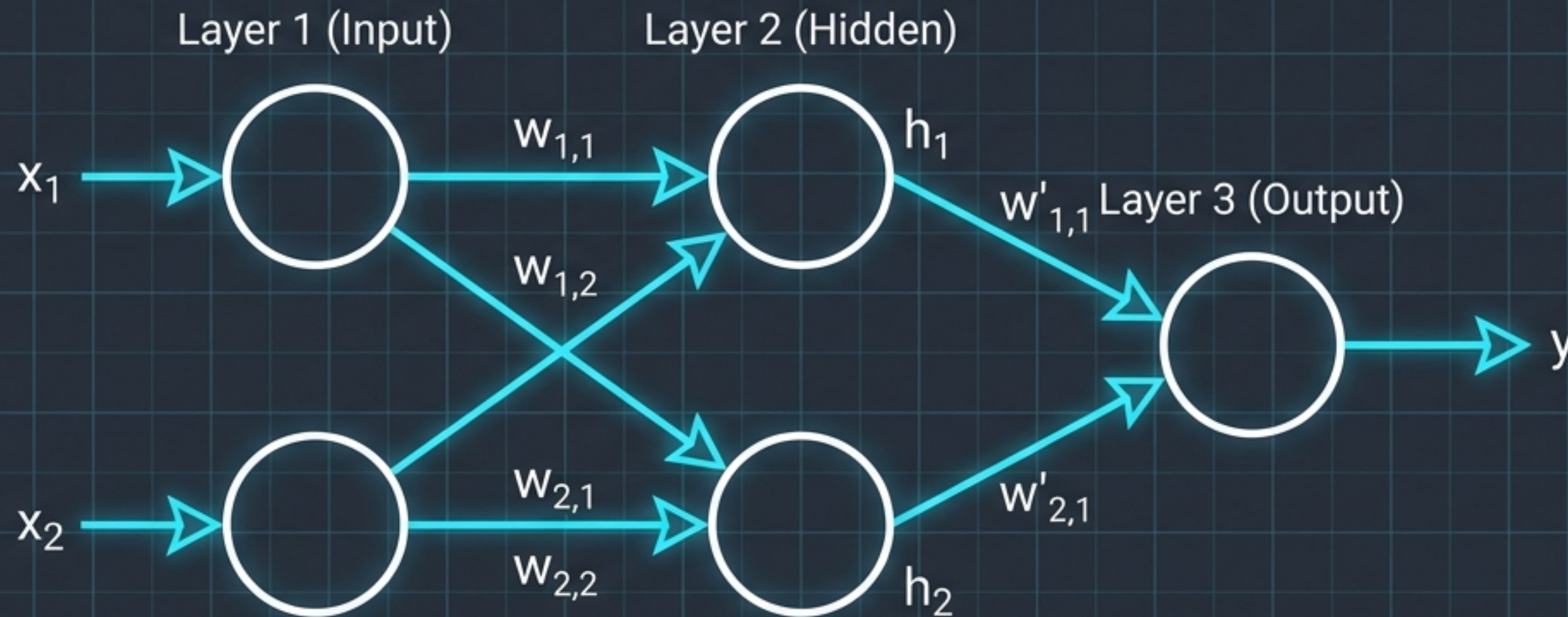


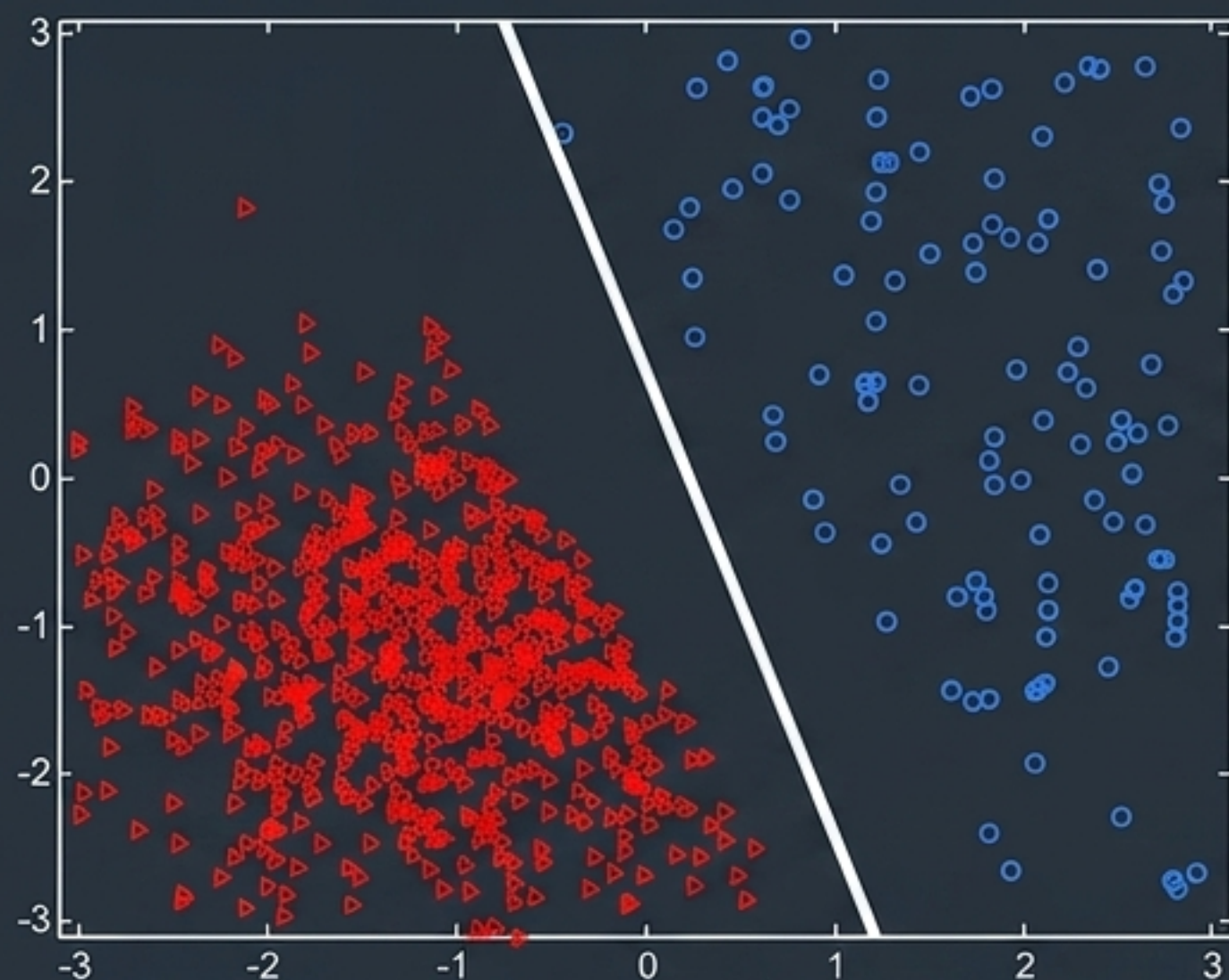
Lab 3 — Wielowarstwowe sieci jednokierunkowe i propagacja wsteczna

Od pojedynczego perceptronu do architektury Feedforward (MLP)

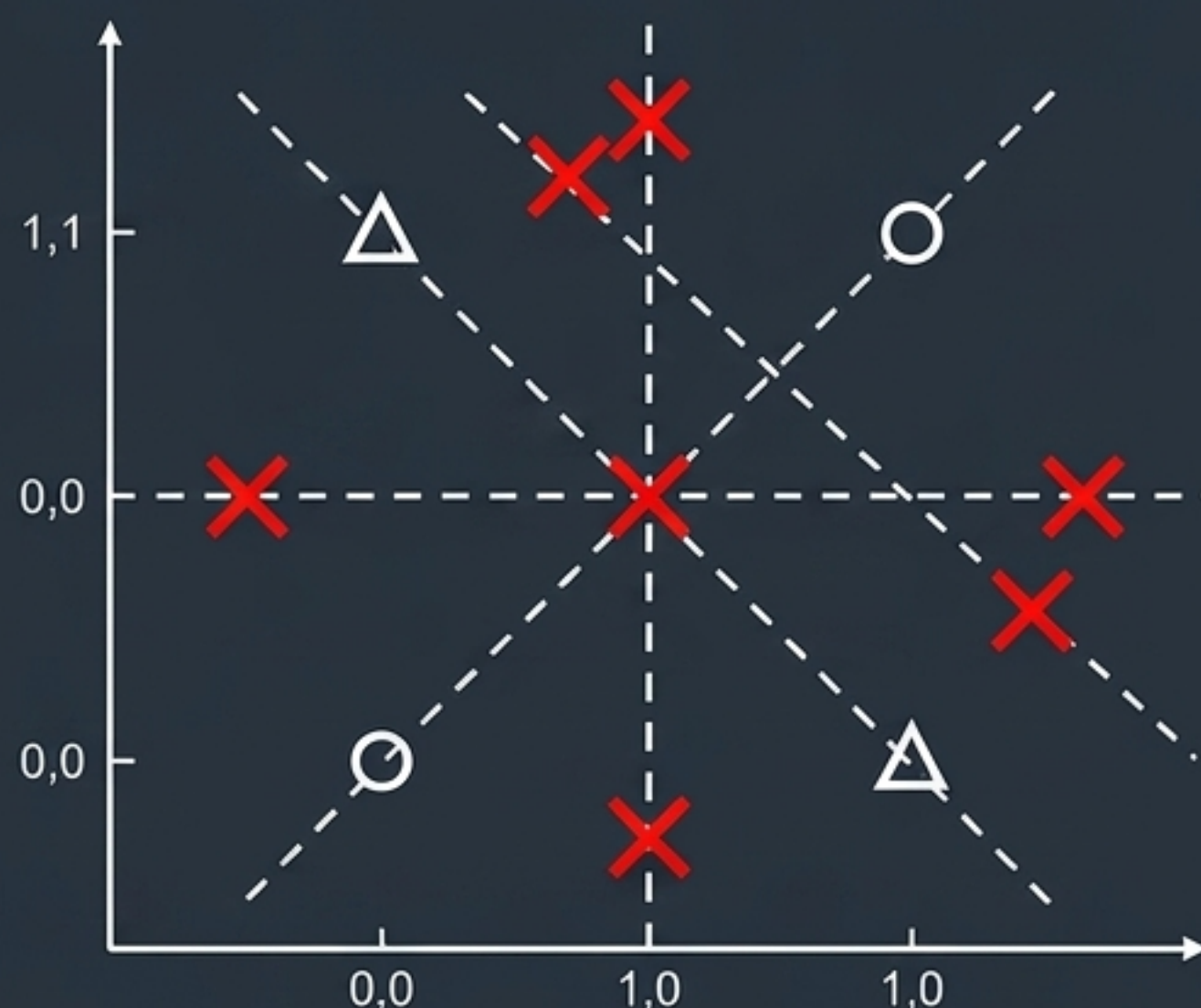


Paradoks XOR: Dlaczego pojedynczy perceptron poległ?

Separowalność liniowa (AND/OR)

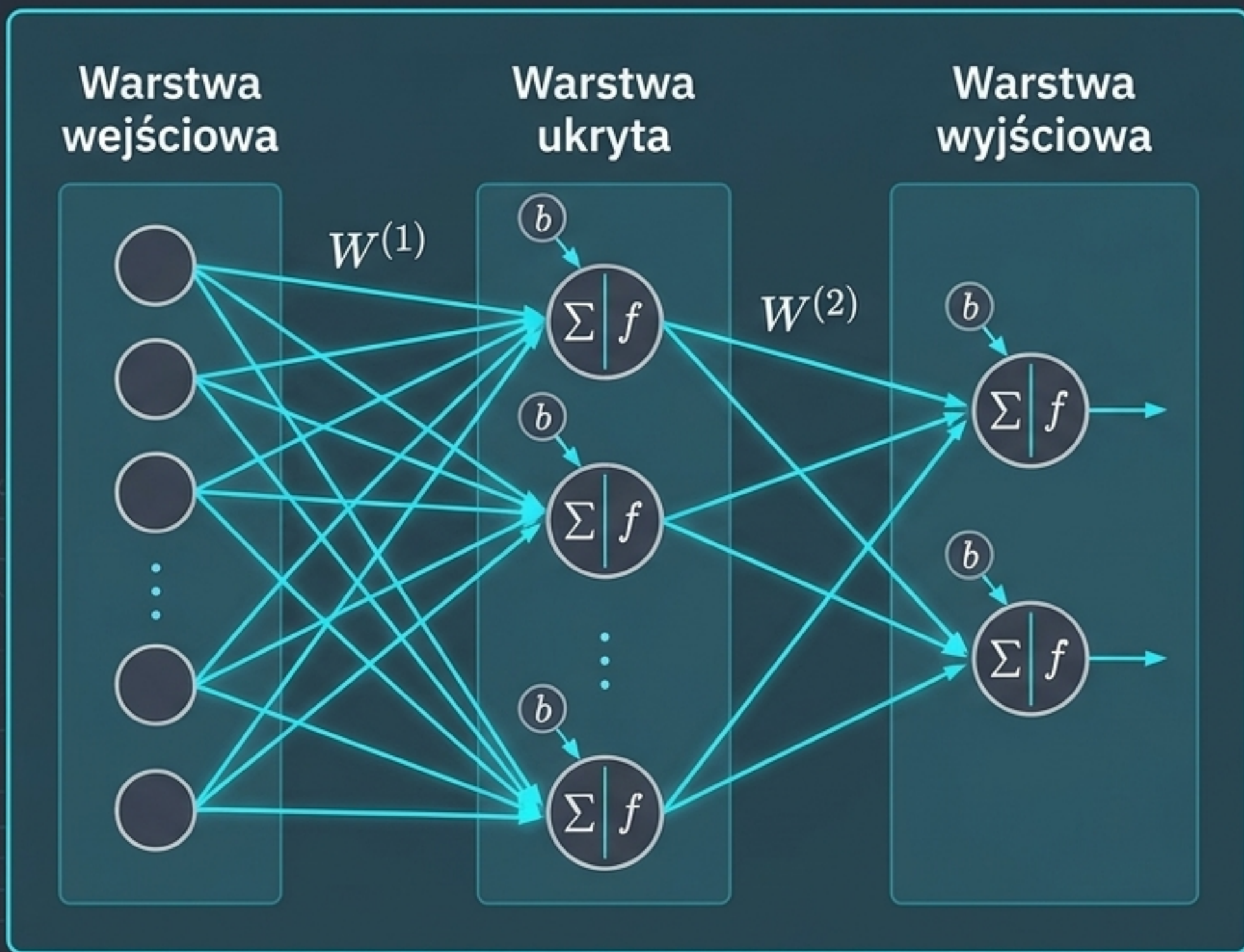


Brak separowalności liniowej (XOR)



Pojedynczy perceptron, bazujący na jednej hiperpłaszczyźnie, potrafi rozdzielić tylko dane liniowo separowalne. Problem XOR (Minsky i Papert, 1969) udowodnił jego matematyczne ograniczenia. **Rozwiązanie?** Dodanie warstw ukrytych, pozwalających na łamanie liniowości.

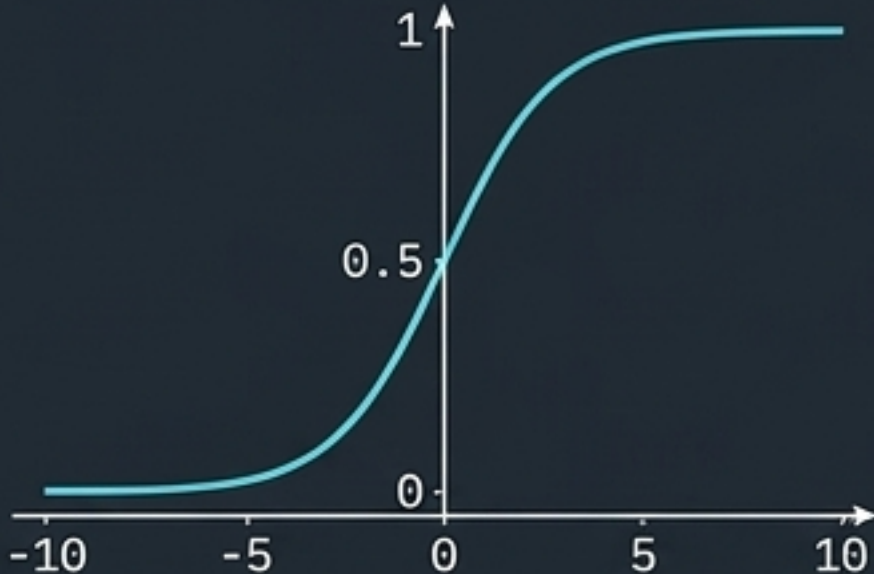
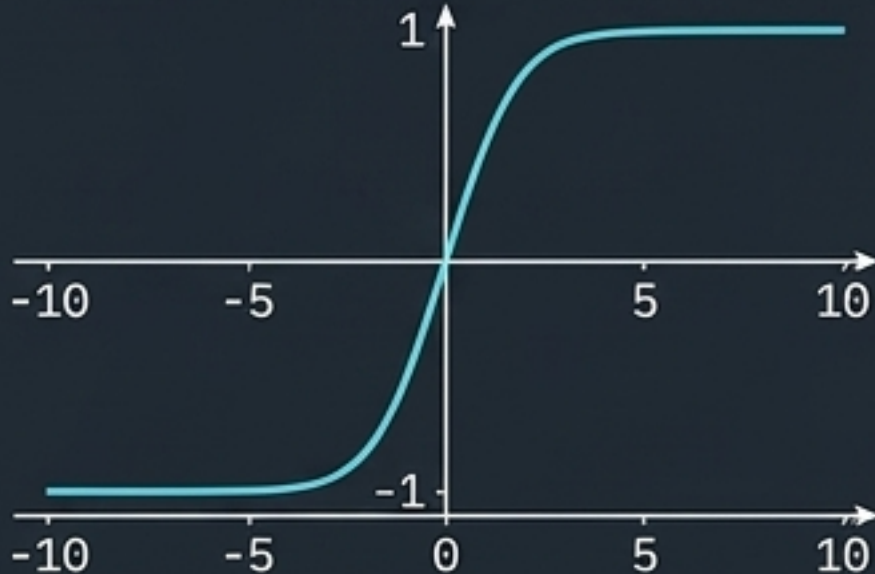
Architektura MLP (Multi-Layer Perceptron)



$$y = f \left(\sum_i w_i x_i + b \right)$$

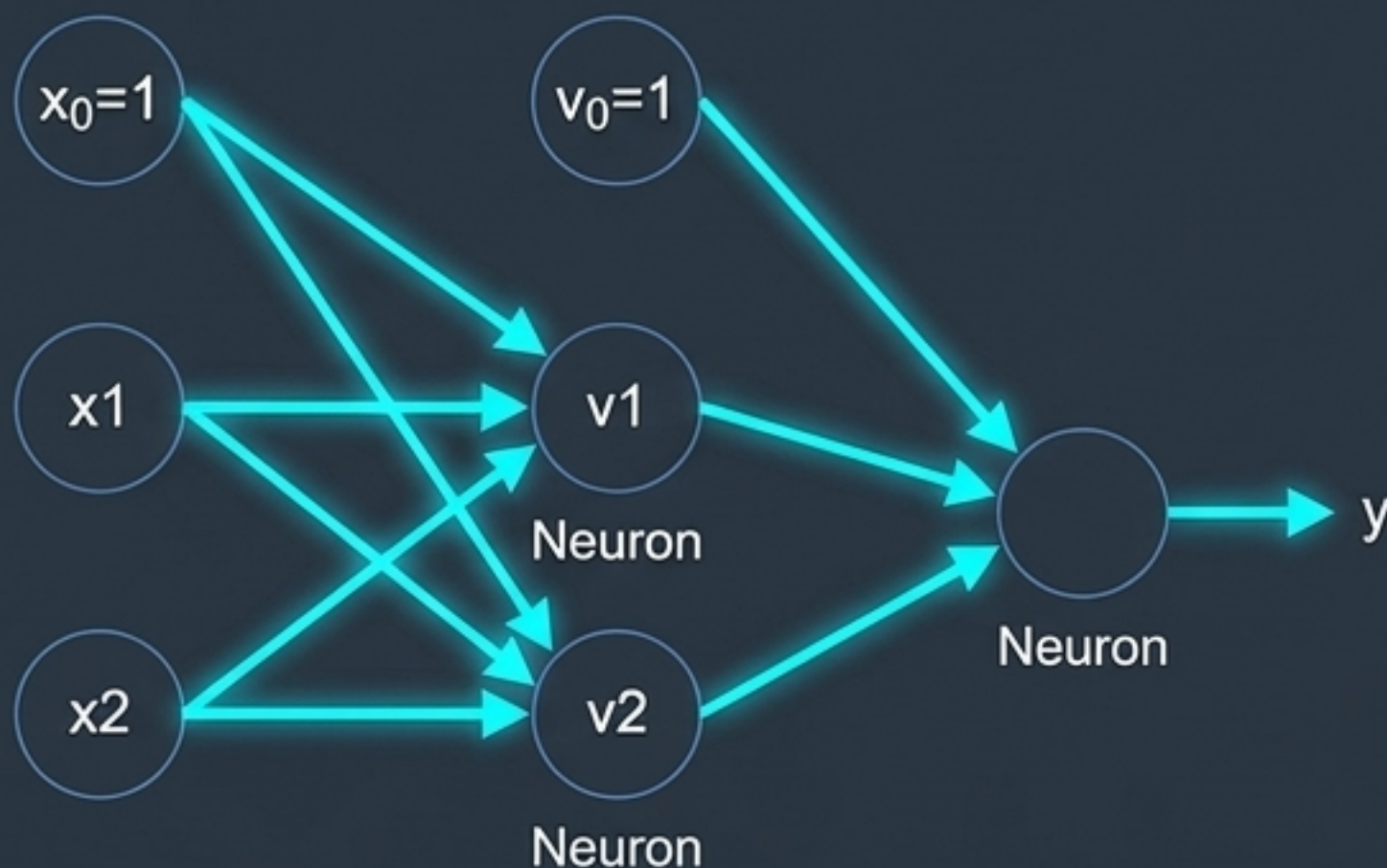
Sygnał płynie wyłącznie w jednym kierunku (Feedforward). Neurony warstwy ukrytej uczą się wewnętrznych, nieliniowych reprezentacji danych.

Przełamanie liniowości: Funkcje aktywacji

	Sigmoid	Tanh
Wykres		
Wzór	$\sigma(x) = \frac{1}{1 + e^{-\beta x}}$	$\tanh(x)$
Pochodna	$\sigma'(x) = \beta \cdot \sigma(x) \cdot (1 - \sigma(x))$	$\tanh'(x) = \beta \cdot (1 - \tanh^2(x))$
Zakres & Cechy	- Zakres: $[0, 1]$ - Dobra do prawdopodobieństw - Problem wygasającego gradientu	- Zakres: $[-1, 1]$ - Wycentrowana w zerze - Szybsza zbieżność

Forward Pass: Przepływ sygnału (Wektory i Macierze)

Topology Diagram



Operacje Macierzowe

Warstwa ukryta:

$$V = f_1(W^{(1)} \cdot X)$$

Warstwa wyjściowa:

$$Y = f_2(W^{(2)} \cdot V)$$

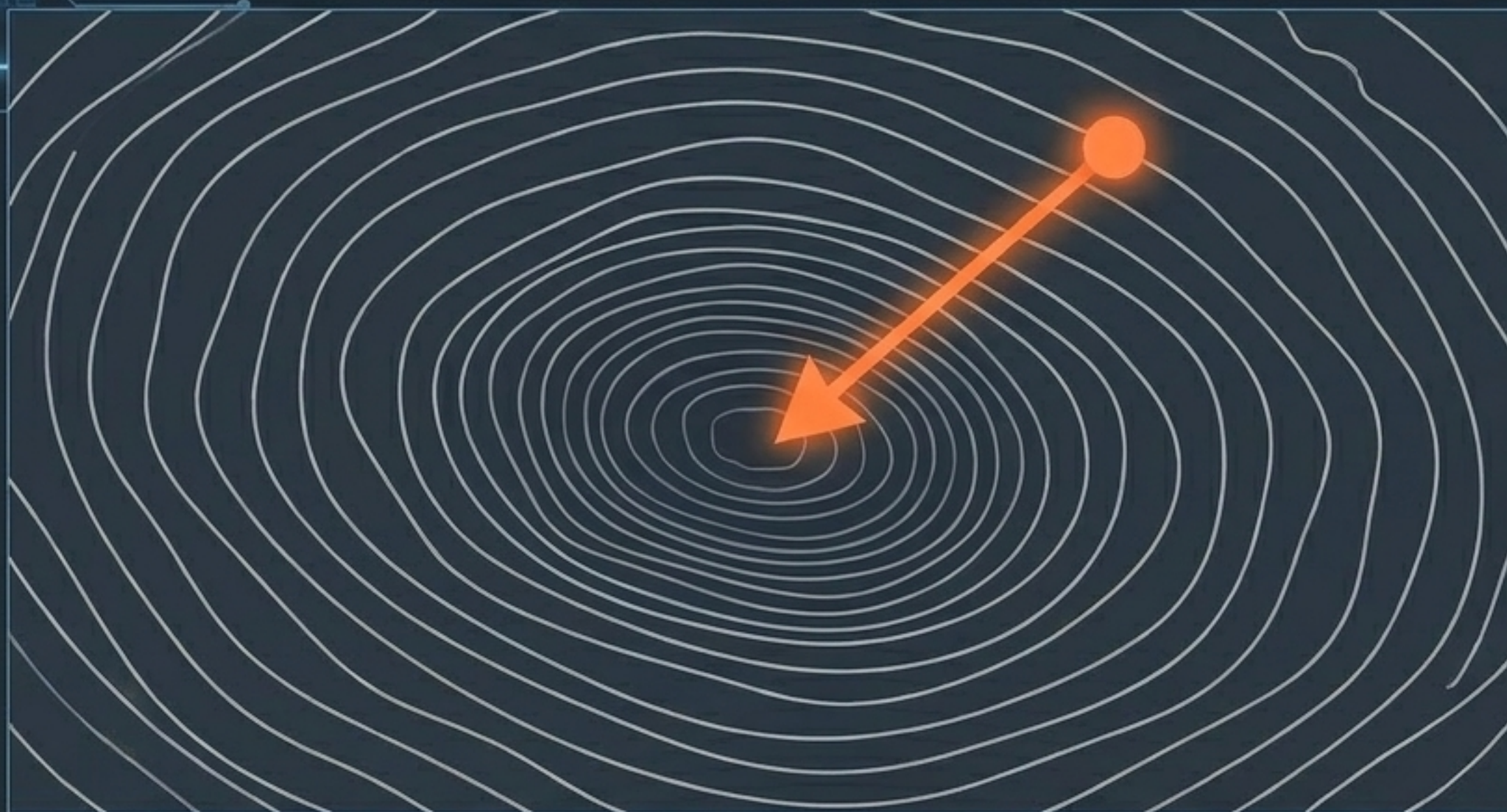
Zamiast pętli 'for' po każdym neuronie, grupujemy sygnały w wektory, a wagi w macierze. Operacje macierzowe drastycznie przyspieszają obliczenia.

Forward Pass w Kodzie (NumPy)

```
1 # Dodane biasu (x0 = 1) do wektora wejściowego
2 X_with_bias = np.insert(X, 0, 1)
3
4 # Obliczenia warstwy ukrytej (np. Tanh)
5 net_h = np.dot(W1, X_with_bias)
6 V = np.tanh(beta * net_h)
7
8 # Dodanie biasu do wyjścia z warstwy ukrytej
9 V_with_bias = np.insert(V, 0, 1)
10
11 # Obliczenia warstwy wyjściowej (np. Sigmoid)
12 net_o = np.dot(W2, V_with_bias)
13 Y = 1 / (1 + np.exp(-beta * net_o))
```

Wektoryzacja w NumPy eliminuje potrzebę ręcznego sumowania.
Wywołanie `np.dot()` załatwia całą algebrę liniową za jednym zamachem.

Intuicja optymalizacji: Gradient Descent

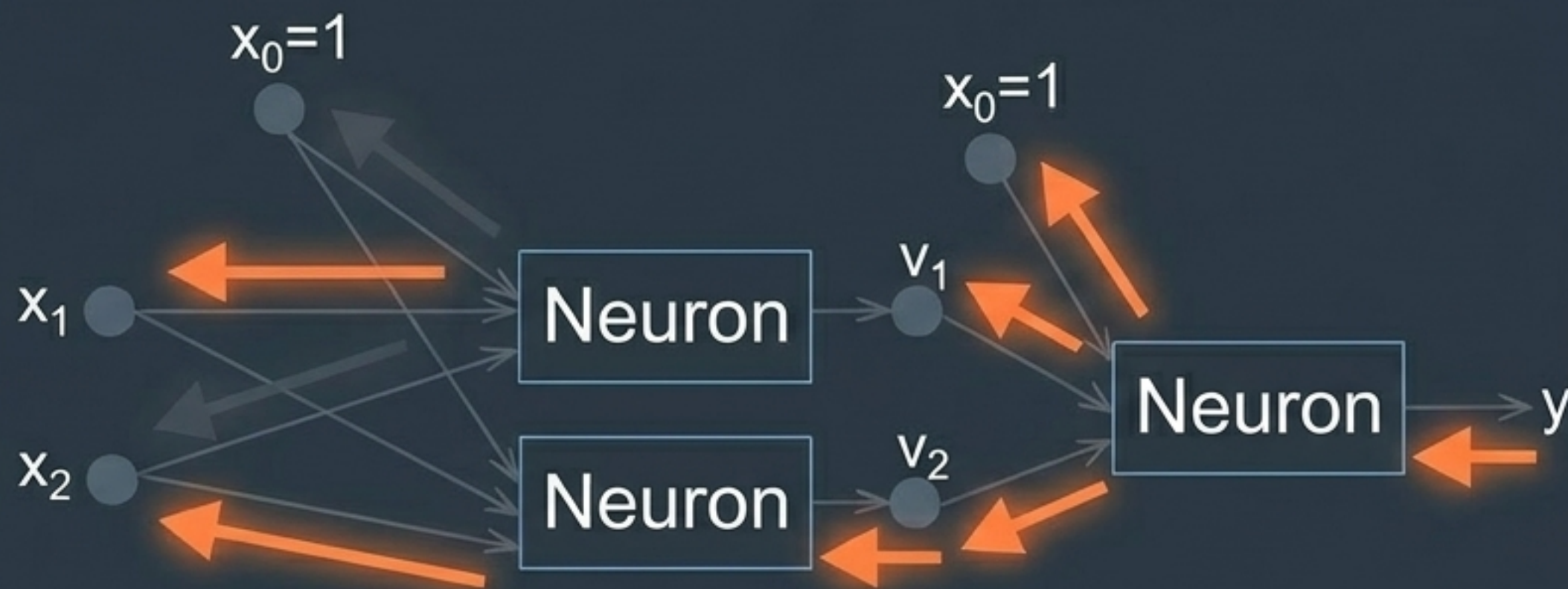


Schodzenie z góry we mgle

Wyobraź sobie, że stoisz na zboczu i nie widzisz całej doliny (powierzchni błędu). Aby zejść do minimum, badasz spadek terenu pod stopami (**gradient**) i robisz krok w dół.

Długość kroku określa Twój **współczynnik uczenia** (η).

Propagacja wsteczna (Reguła Łańcuchowa)



Aparat Matematyczny

Funkcja błędu (MSE):

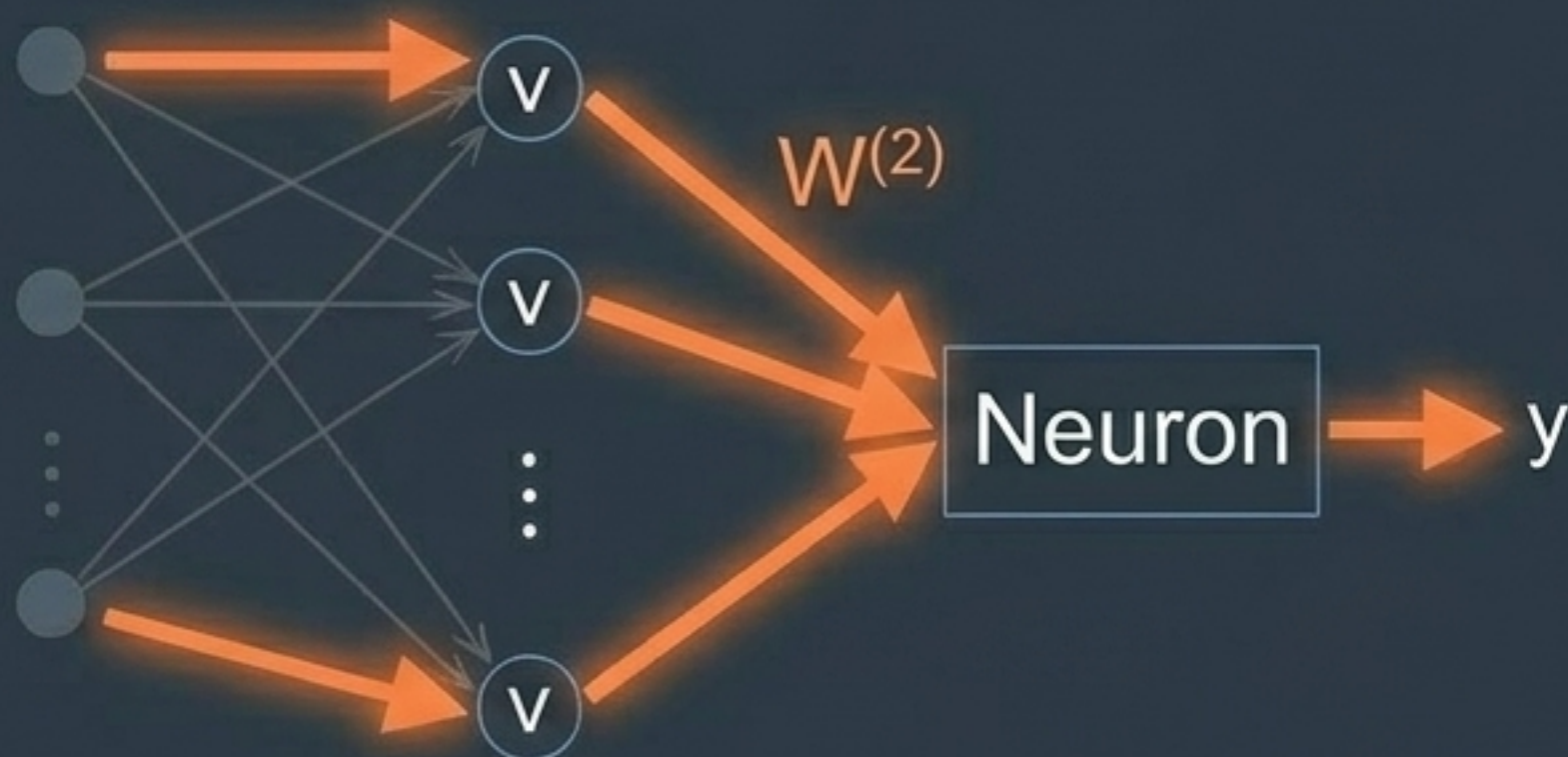
$$E = \frac{1}{2} (y_{\text{oczekiwane}} - y_{\text{obliczone}})^2$$

Reguła łańcuchowa (Chain Rule):

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial \text{net}} \cdot \frac{\partial \text{net}}{\partial w}$$

Algorytm rozdziela całkowity błąd z powrotem na poszczególne wagi. Ponieważ funkcje aktywacji są różniczkowalne, możemy precyzyjnie określić, jak zmiana każdej wagi wpłynie na końcowy błąd.

Wyprowadzenie Gradientu (Warstwa Wyjściowa $W^{(2)}$)



$$\frac{\partial E}{\partial W^{(2)}} = \frac{\partial E}{\partial y} \cdot \frac{\partial y}{\partial net} \cdot \frac{\partial net}{\partial W^{(2)}}$$

$$\frac{\partial E}{\partial y} = -(d - y) \text{ (Gdzie } d \text{ to wartość oczekiwana)}$$

$$\frac{\partial y}{\partial net} = f'_2(net) \text{ (Pochodna funkcji aktywacji)}$$

$$\frac{\partial net}{\partial W^{(2)}} = V \text{ (Sygnał wejściowy z warstwy ukrytej)}$$

Definiujemy lokalny błąd (Delta): $\delta(2) = -(d - y) \cdot f'_2(net)$
Zatem gradient wynosi: $\delta(2) \cdot V$

Wyprowadzenie Gradientu (Warstwa Ukryta $W^{(1)}$)

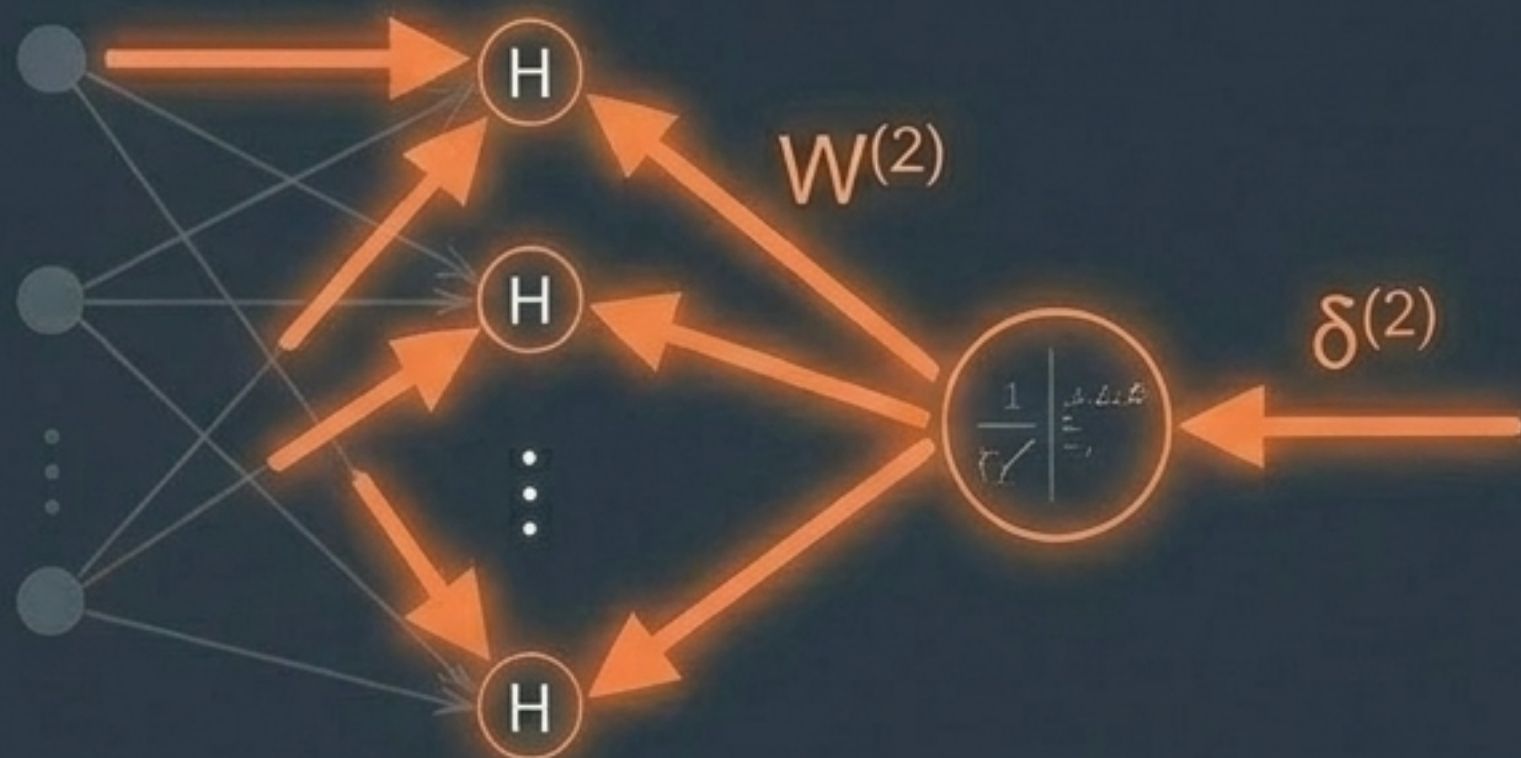
Aparat Matematyczny

Błąd propagowany wstecz:

$$\delta^{(1)} = (\delta^{(2)} \cdot W^{(2)}) \cdot f'_1(net_{\text{hidden}})$$

Gradient-t dla wag ukrytych:

$$\frac{\partial E}{\partial W^{(1)}} = \delta^{(1)} \cdot X$$



Prawdziwa moc Backpropagation: Neurony ukryte nie mają bezpośredniej wartości oczekiwanej. Otrzymują one 'proporcjonalną winę' za błąd od neuronów wyjściowych, z którymi są połączone.

Aktualizacja Wag (Reguła Delta)

Aparat Matematyczny

$$W_{nowe} = W_{stare} - \eta \cdot \frac{\partial E}{\partial W}$$

Implementacja Kodowa

```
1 # Aktualizacja wag z użyciem
2 # współczynnika uczenia (eta)
3
4 W2 = W2 - eta * gradient_W2
5 W1 = W1 - eta * gradient_W1
6
```

Mając wyliczone gradienty, przesuwamy wagi w kierunku przeciwnym do wektora gradientu (w dół zbocza). Proces ten powtarzamy iteracyjnie (epoki), aż błąd spadnie poniżej ustalonego progu.

Dynamika Uczenia: Online vs Batch

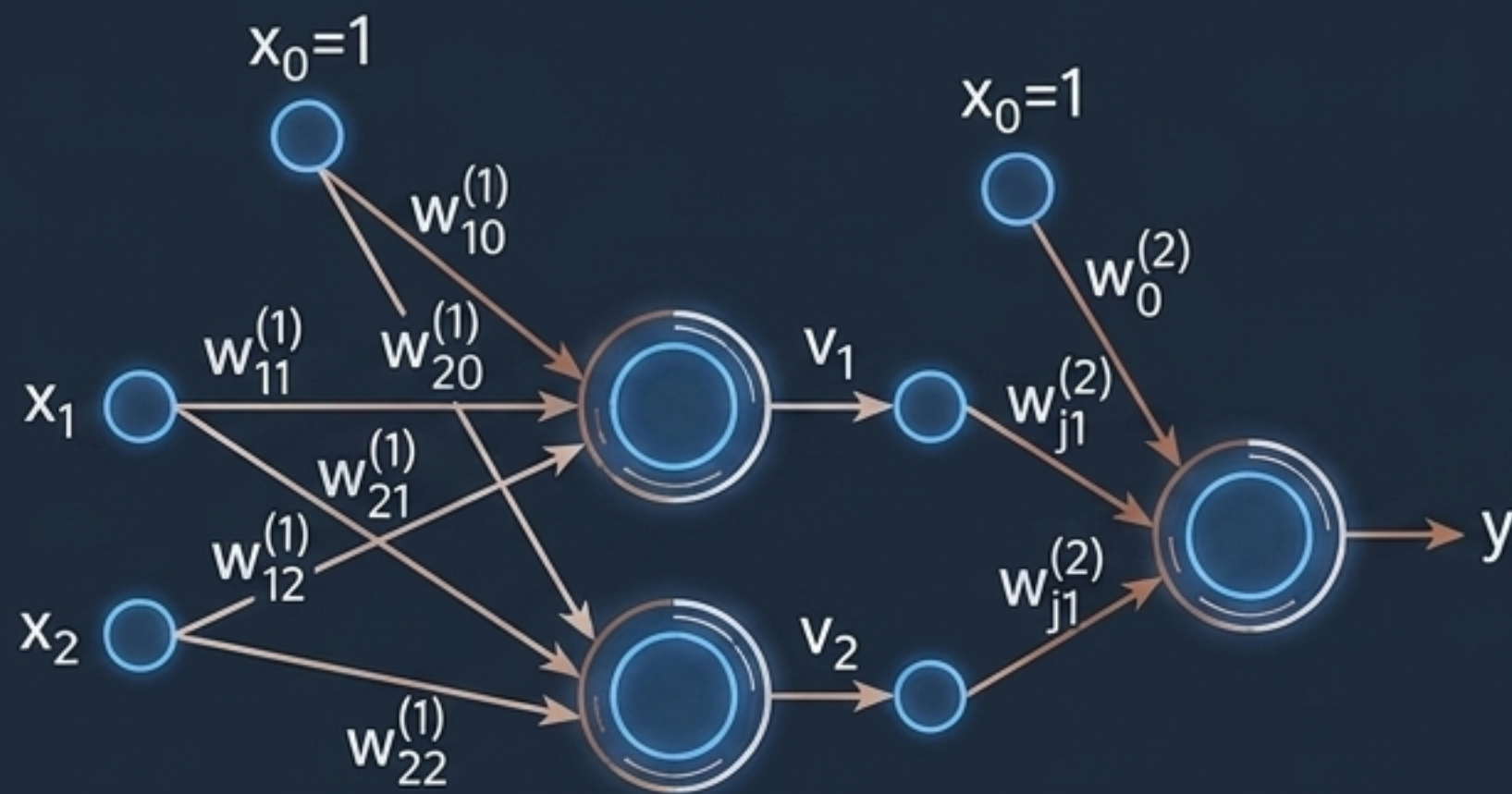
Uczenie Online (Stochastyczne)

- Kiedy aktualizacja? Po każdej pojedynczej próbie (wzorcu).
- Zalety: Szum pomaga uciec z minimów lokalnych.
- Wady: Wymaga pętli w Pythonie, wolniejsze obliczeniowo. Trajektoria jest poszarpana.

Uczenie Batch (Wsadowe)

- Kiedy aktualizacja? Raz na epokę (po uśrednieniu gradientów dla całego zbioru).
- Zalety: Gładki, stabilny spadek gradientu. W pełni wykorzystuje wektoryzację macierzy w NumPy (bardzo szybkie).
- Wady: Może utknąć w płytkich minimach lokalnych.

Zadanie 1 – Implementacja Forward Pass

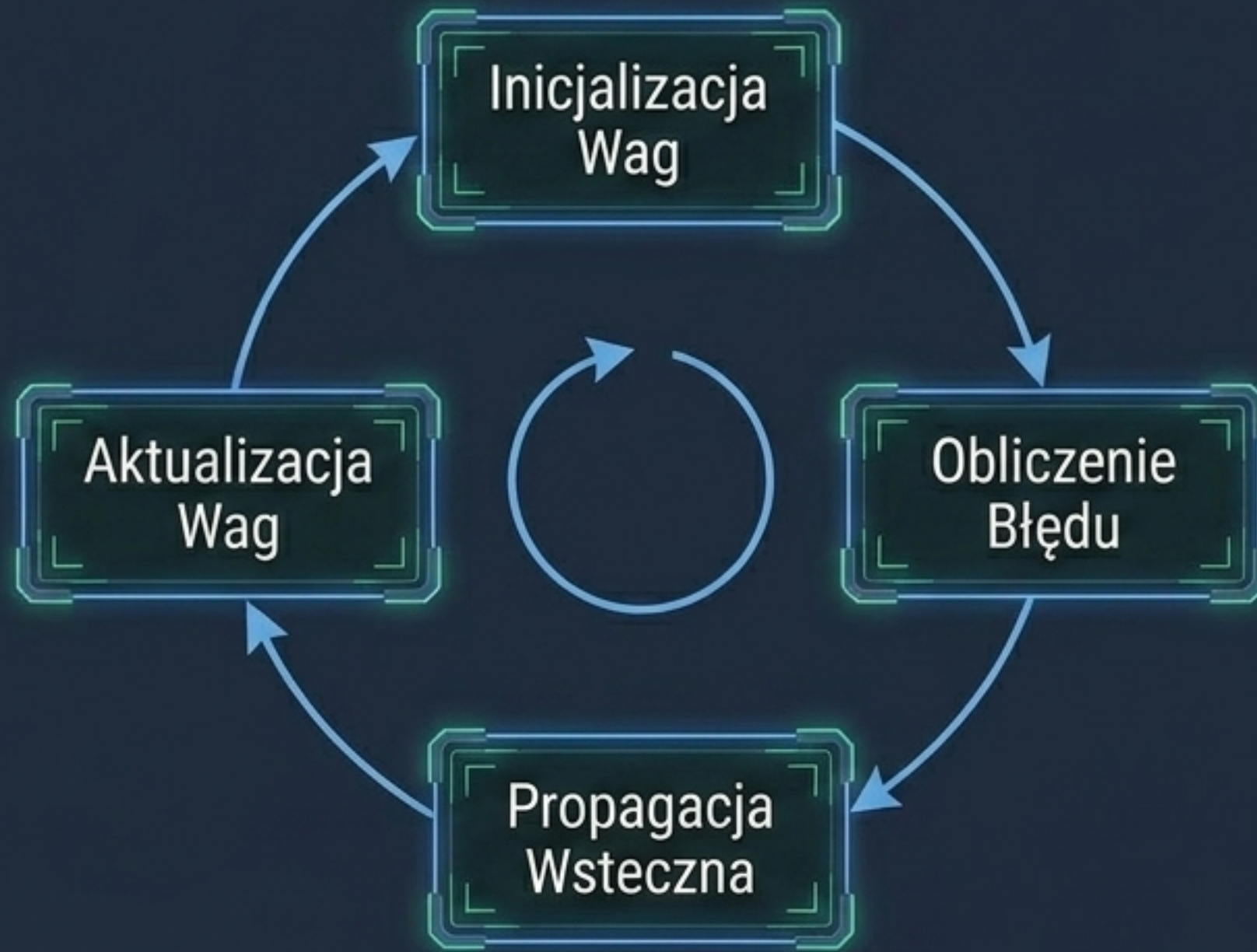


```
def mlp(x, w1, w2, beta):  
    pass
```

Wymagania

- ✓ - Architektura 2-2-1.
- ✓ - Warstwa ukryta: bipolarna funkcja **tanh**.
- ✓ - Warstwa wyjściowa: unipolarna funkcja **sigmoid**.
- ✓ - Reprezentacja wektorowo-macierzowa (**Numpy**).
- ✓ - **PROTIP**: Nie zapomnij o dodaniu sygnału biasu ($x_0=1$, $v_0=1$) na wejściu każdej warstwy!

Zadanie 2 — Zamknięcie pętli: Rozwiązanie XOR

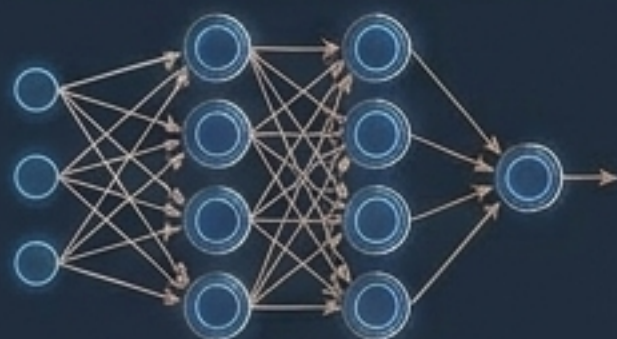


Task Box

- Zbuduj pętlę uczącą dla funkcji `mlp()` z Zadania 1.
- Zbiór uczący: Brama logiczna XOR.
- Zaimplementuj i przetestuj dwa warianty: **Uczenie Online** oraz **Uczenie Batch**.
- Obserwacja: Porównaj szybkość zbieżności i stabilność obu metod.

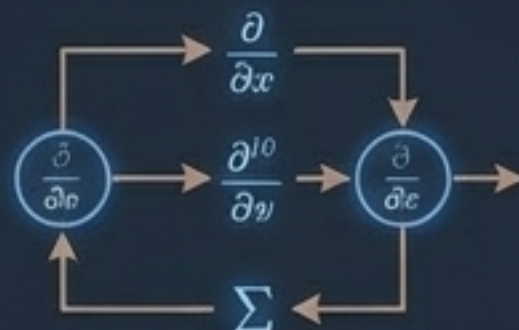
Synteza i zapowiedź: Co dalej?

1. Architektura



MLP z warstwami ukrytymi łamie ograniczenia liniowe (rozwiązuje XOR).

2. Matematyka



Propagacja wsteczna efektywnie rozdziela błąd używając reguły łańcuchowej i funkcji różniczkowalnych.

3. Kod

```
np.dot(W, X) - [⋮ × ⋮]
```

Wektoryzacja w NumPy przenosi ciężar obliczeń z wolnych pętli na zoptymalizowaną algebrę macierzową.

Lab 4: Sieć działa, ale co się stanie, gdy dodamy 10, 50 lub 100 warstw?

Na następnych zajęciach: Problem znikającego gradientu, optymalizatory adaptacyjne i głębokie uczenie (Deep Learning).